

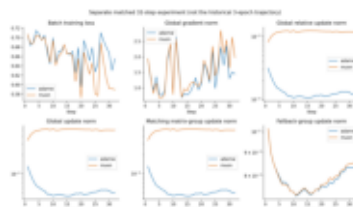
AdamW versus Muon: DistilBERT on SST-2

1. Which optimizer performed better, and by what criterion? Criterion: validation accuracy and loss after 3 matched epochs on SST-2 (67,349 train / 872 validation, batch size 32, seed 42, constant LR). AdamW (lr 2e-5 everywhere) reached final/best accuracy 90.83% (loss 0.26245, epoch 3); validation loss stayed narrow across epochs (0.2577, 0.2873, 0.2645). Muon (lr 0.003 on the 37 hidden weight matrices, AdamW lr 2e-5 fallback elsewhere) reached final accuracy 84.40% (loss 0.39393, epoch 3), best at epoch 1 (87.04%, loss 0.36956); its validation loss rose to 0.5706 at epoch 2 even as training loss kept falling, then eased at epoch 3 — a constant, undecayed learning rate may make mid-training validation loss more sensitive to a fixed step size, though a single run cannot confirm this. By final/best accuracy, AdamW performed better here. Muon's matrix LR was fixed by a predeclared stability check over {0.001, 0.003, 0.01}, selecting 0.003 as the smallest stable rate rather than the best-scoring one (0.01 reached a lower holdout loss); AdamW's 2e-5 came from common practice, so the two were not comparably tuned (log in `SUPPLEMENTARY\_ANALYSIS.md`). A separate matched 32-step diagnostic (same init/batches/RNG; not a reconstruction of the full run — see figure below) suggests why: Muon's matrix-group update norm rises over ~5 steps then plateaus near 0.49, while AdamW's starts near 0.13 and decays toward 0.05; matrix-group gradient norms overlap for both with no sustained divergence, so the gap reflects the update rule itself — Muon's orthogonalized step versus AdamW's adaptive scaling — not gradient magnitude (ranges in `SUPPLEMENTARY\_ANALYSIS.md`).

2. Which solution appears flatter? We perturbed each best checkpoint on 256 shared validation examples along 8 paired random directions per tensor, scaled to epsilon times each tensor's own norm, and measured the mean loss increase — a curvature proxy: a flatter minimum tolerates larger perturbations with less loss increase. At epsilon=0.1, AdamW's mean increase was 0.00741 versus 0.00435 for Muon, suggesting Muon's checkpoint is locally flatter. But the paired Muon-minus-AdamW 95% interval is [-0.00976, 0.00365] — it includes zero, so the difference is not distinguishable from noise here. The checkpoints are also from different epochs (AdamW 3, Muon 1) with different baseline losses (0.2253 vs 0.3199). Eight random directions do not identify worst-case curvature, and the per-tensor scaling is not fully reparameterization-invariant, so these numbers describe sensitivity along sampled directions only, not a worst-case bound.

3. How reliable are these conclusions? Limited. Each optimizer used a single seed, so there is no run-to-run variance estimate. Historical per-step norms were never recorded for the original runs; the 32-step diagnostic is a separate, short, matched proxy for early-step dynamics, not a reconstruction of the full trajectory, and should not be read as explaining the final accuracy gap. Both optimizers use weight_decay=0.01 as decoupled decay; Muon's matrix-group lr (0.003) makes its realized per-step decay ~150x AdamW's (lr 2e-5), confounding the update-rule and regularization differences. Muon's full run also took 3621s versus AdamW's 1603s (~2.26x) on this M4/MPS machine with an unfused implementation — a hardware/implementation-specific cost, not a general throughput claim (details in `SUPPLEMENTARY\_ANALYSIS.md`).

4. What would you change in a larger study? Use multiple seeds to quantify variance in accuracy, loss, and sharpness. Record real per-step norms during the full runs, not a short proxy. Evaluate sharpness at matched checkpoint epochs. Tune AdamW's rate with the same predeclared procedure used for Muon and match effective weight decay (lr x weight_decay), not the nominal coefficient. Time both optimizers with a fused Muon implementation before drawing throughput conclusions.



32-step matched diagnostic (full-resolution figure: deliverables/optimizer_diagnostics.png)

Sources: saved runs and CSV tables in deliverables/; figures provided separately.